

Terraform Task - Sudir

Task Description:

Launch Linux EC2 instances in two regions using a single Terraform file.

Techstacks needs to be used :

- AWS EC2
- Terraform
- AWS CLI

How do I submit my work?

- Push all your work files to GitHub (Code & O/P screenshot images must).
- Submit your URLs in the portal.

Terms and Conditions?

- You agree to not share this confidential document with anyone.
- You agree to open-source your code (it may even look good on your profile!). Do not mention our company's name anywhere in the code.
- We will never use your source code under any circumstances for any commercial purposes; this is just a basic assessment task.

NOTE: Any violation of Terms and conditions is strictly prohibited. You are bound to adhere to it

1. Install Terraform

On Amazon Linux:

`sudo dnf install -y terraform`

```
root@ip-172-31-41-142 ~]# cat /etc/os-release
AME="Amazon Linux"
ERSION="2023"
D="aman"
D LIKE="fedora"
ERSION_ID="2023"
LATFORM_ID="platform:al2023"
RETTY_NAME="Amazon Linux 2023.11.20260526"
NSI_COLOR="0;33"
PE_NAME="cpe:2.3:o:amazon:amazon_linux:2023"
OME_URL="https://aws.amazon.com/linux/amazon-linux-2023/"
OCUMENTATION_URL="https://docs.aws.amazon.com/linux/"
UPPORT_URL="https://aws.amazon.com/premiumsupport/"
UG_REPORT_URL="https://github.com/amazonlinux/amazon-linux-2023"
ENDOR_NAME="AWS"
ENDOR_URL="https://aws.amazon.com/"
UPPORT_END="2029-06-30"
root@ip-172-31-41-142 ~]# sudo dnf install -y dnf-plugins-core
amazon Linux 2023 Kernel Livepatch repository
ackage dnf-plugins-core-4.3.0-13.amzn2023.0.6.noarch is already installed.
ependencies resolved.
othing to do.
omplete!
root@ip-172-31-41-142 ~]# sudo dnf config-manager --add-repo https://rpm.releases.hashicorp.com/AmazonLinux/hashicorp.repo
dding repo from: https://rpm.releases.hashicorp.com/AmazonLinux/hashicorp.repo
root@ip-172-31-41-142 ~]# sudo dnf install -y terraform
ashicorp Stable - x86_64
ependencies resolved.
```

Package	Architecture	Version
Installing:		
terraform	x86_64	1.15.5-1
Installing dependencies:		
git	x86_64	2.50.1-1.amzn2023.0.1
git-core	x86_64	2.50.1-1.amzn2023.0.1
git-core-doc	noarch	2.50.1-1.amzn2023.0.1
perl-Error	noarch	1:0.17030-2.amzn2023.0.1
perl-File-Find	noarch	1.37-477.amzn2023.0.8
perl-Git	noarch	2.50.1-1.amzn2023.0.1
perl-TermReadKey	x86_64	2.38-9.amzn2023.0.3
perl-lib	x86_64	0.65-477.amzn2023.0.8
Transaction Summary		
Install 9 Packages		

terraform version

```
Installed:
git-2.50.1-1.amzn2023.0.1.x86_64      git-core-2.50.1-1.amzn2023.0.1.x86_64
perl-File-Find-1.37-477.amzn2023.0.8.noarch  perl-Git-2.50.1-1.amzn2023.0.1.noarch
terraform-1.15.5-1.x86_64

Complete!
[root@ip-172-31-41-142 ~]# terraform version
Terraform v1.15.5
on linux_amd64
[root@ip-172-31-41-142 ~]# aws --version
aws-cli/2.33.15 Python/3.9.25 Linux/6.1.172-216.329.amzn2023.x86_64 source/x86_64.amzn.2023
```

2. Verify AWS Access

role is attached to the EC2:

aws sts get-caller-identity

```
root@ip-172-31-41-142 ~]# aws sts get-caller-identity
{
  "UserId": "AROAZMRYQSGKI46762R7K:i-02466e1c625ac40cb",
  "Account": "645437362580",
  "Arn": "arn:aws:sts::645437362580:assumed-role/sudir/i-02466e1c625ac40cb"
}
```

3. Create Project Directory

```
[root@ip-172-31-41-142 ~]# ll
total 0
drwxr-xr-x. 3 root root 91 Jun  4 09:15 terraform-ec2
```

4. Create main.tf

Replace the AMI IDs with valid Amazon Linux AMIs for your regions.

```
terraform {
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}
```

```
# Region 1
provider "aws" {
  region = "ap-south-1"
}
```

```
# Region 2
provider "aws" {
  alias = "us_east"
  region = "us-east-1"
}
```

```
# EC2 in Mumbai
resource "aws_instance" "mumbai_ec2" {
  ami      = "ami-0685bcc683dadb6b9"
  instance_type = "t2.micro"

  tags = {
    Name = "Terraform-Mumbai-EC2"
  }
}
```

```
# EC2 in Virginia
resource "aws_instance" "virginia_ec2" {
  provider   = aws.us_east
  ami        = "ami-00e801948462f718a"
  instance_type = "t2.micro"

  tags = {
    Name = "Terraform-Virginia-EC2"
  }
}
```

```
output "mumbai_instance_id" {
  value = aws_instance.mumbai_ec2.id
}
```

```
output "virginia_instance_id" {
  value = aws_instance.virginia_ec2.id
}
```

```
root@ip-172-31-41-142 terraform-ec2]# ll
total 16
-rw-r--r-- 1 root root 784 Jun  4 09:13 main.tf
```

5. Get AMI IDs

Mumbai:

```
aws ssm get-parameter \  
--name /aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64 \  
--region ap-south-1
```

ami-0685bcc683dadb6b9

Virginia:

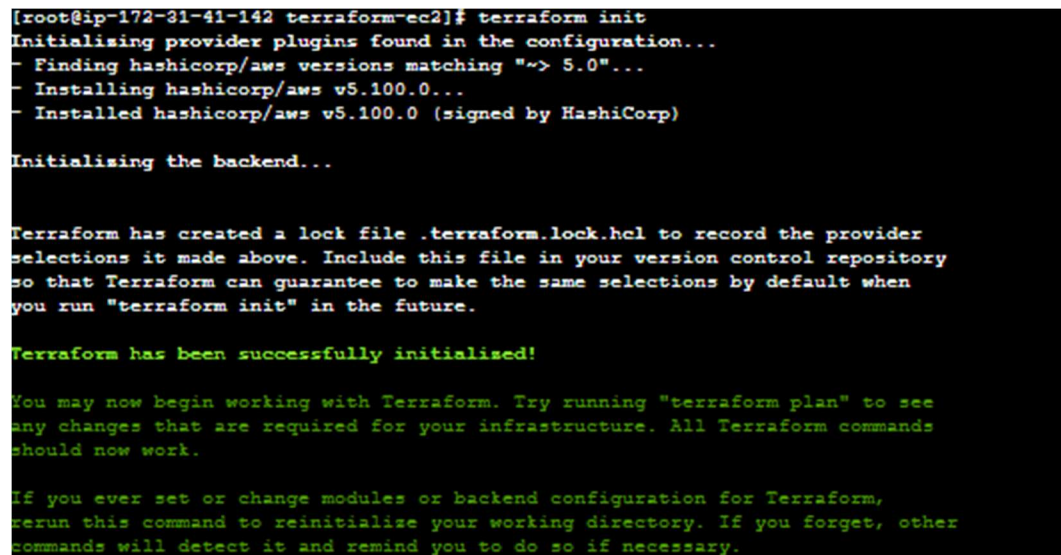
```
aws ssm get-parameter \  
--name /aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64 \  
--region us-east-1
```

ami-00e801948462f718a

Copy the AMI values into main.tf.

6. Initialize Terraform

terraform init

A terminal window with a black background and green text. The output of the 'terraform init' command is shown. It starts with '[root@ip-172-31-41-142 terraform-ec2]# terraform init', followed by 'Initializing provider plugins found in the configuration...'. A list of actions follows: '- Finding hashicorp/aws versions matching "~> 5.0"...', '- Installing hashicorp/aws v5.100.0...', and '- Installed hashicorp/aws v5.100.0 (signed by HashiCorp)'. Then it says 'Initializing the backend...'. A paragraph explains that Terraform has created a lock file '.terraform.lock.hcl' to record provider selections. Another paragraph states 'Terraform has been successfully initialised!'. A final paragraph encourages the user to begin working with Terraform and try running 'terraform plan'.

```
[root@ip-172-31-41-142 terraform-ec2]# terraform init  
Initializing provider plugins found in the configuration...  
- Finding hashicorp/aws versions matching "~> 5.0"...  
- Installing hashicorp/aws v5.100.0...  
- Installed hashicorp/aws v5.100.0 (signed by HashiCorp)  
  
Initializing the backend...  
  
Terraform has created a lock file .terraform.lock.hcl to record the provider  
selections it made above. Include this file in your version control repository  
so that Terraform can guarantee to make the same selections by default when  
you run "terraform init" in the future.  
  
Terraform has been successfully initialised!  
  
You may now begin working with Terraform. Try running "terraform plan" to see  
any changes that are required for your infrastructure. All Terraform commands  
should now work.  
  
If you ever set or change modules or backend configuration for Terraform,  
rerun this command to reinitialize your working directory. If you forget, other  
commands will detect it and remind you to do so if necessary.
```

7. Validate

terraform validate

```
[root@ip-172-31-41-142 terraform-ec2]# terraform validate
Success! The configuration is valid.
```

8. Review Changes

terraform plan

also you can use terraform plan -out ec2 -> to save the plan

```
lan: 2 to add, 0 to change, 0 to destroy.

Changes to Outputs:
+ mumbai_instance_id   = (known after apply)
+ virginia_instance_id = (known after apply)
```

9. Launch EC2 Instances

terraform apply

```
Enter a value: yes

aws_instance.mumbai_ec2: Creating...
aws_instance.virginia_ec2: Creating...
aws_instance.mumbai_ec2: Still creating... [00m10s elapsed]
aws_instance.virginia_ec2: Still creating... [00m10s elapsed]
aws_instance.mumbai_ec2: Creation complete after 13s [id=i-08d61f9a2ec1bfb6e6]
aws_instance.virginia_ec2: Still creating... [00m20s elapsed]
aws_instance.virginia_ec2: Still creating... [00m30s elapsed]
aws_instance.virginia_ec2: Creation complete after 36s [id=i-0c7e8495213de762d]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.

Outputs:

mumbai_instance_id = "i-08d61f9a2ec1bfb6e6"
virginia_instance_id = "i-0c7e8495213de762d"
```

10. Verify EC2 Instances

Mumbai:

```
aws ec2 describe-instances \
--region ap-south-1 \
--query 'Reservations[*].Instances[*].[InstanceId,State.Name]' \
--output table
```

```
[root@ip-172-31-41-142 terraform-ec2]# aws ec2 describe-instances \
--region ap-south-1 \
--query 'Reservations[*].Instances[*].[InstanceId,State.Name]' \
--output table
-----
| DescribeInstances |
+-----+
| i-08d61faec1bfb6e6 | running |
| i-02466e1c625ac40cb | running |
+-----+

```

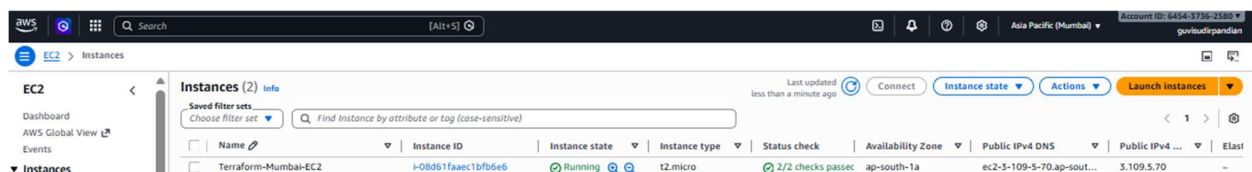
Virginia:

```
aws ec2 describe-instances \
--region us-east-1 \
--query 'Reservations[*].Instances[*].[InstanceId,State.Name]' \
--output table
```

```
[root@ip-172-31-41-142 terraform-ec2]# aws ec2 describe-instances \
--region us-east-1 \
--query 'Reservations[*].Instances[*].[InstanceId,State.Name]' \
--output table
-----
| DescribeInstances |
+-----+
| i-0c7e8495213de762d | running |
+-----+

```

mumbai



Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	Public IPv4 ...	Elasti
Terraform-Mumbai-EC2	i-08d61faec1bfb6e6	Running	t2.micro	2/2 checks passed	ap-south-1a	ec2-3-109-5-70.ap-sout...	3.109.5.70	-

virginia

unable to take screenshot guvi user doesn't access

11. Destroy

terraform destroy

```
Do you really want to destroy all resources?
Terraform will destroy all your managed infrastructure, as shown above.
There is no undo. Only 'yes' will be accepted to confirm.

Enter a value: yes

vs_instance.mumbai_ec2: Destroying... [id=i-08d61faaec1bfb6e6]
vs_instance.virginia_ec2: Destroying... [id=i-0c7e8495213de762d]
vs_instance.mumbai_ec2: Still destroying... [id=i-08d61faaec1bfb6e6, 00m10s elapsed]
vs_instance.virginia_ec2: Still destroying... [id=i-0c7e8495213de762d, 00m10s elapsed]
vs_instance.mumbai_ec2: Still destroying... [id=i-08d61faaec1bfb6e6, 00m20s elapsed]
vs_instance.virginia_ec2: Still destroying... [id=i-0c7e8495213de762d, 00m20s elapsed]
vs_instance.mumbai_ec2: Destruction complete after 30s
vs_instance.virginia_ec2: Still destroying... [id=i-0c7e8495213de762d, 00m30s elapsed]
vs_instance.virginia_ec2: Destruction complete after 32s

Destroy complete! Resources: 2 destroyed.
root@ip-172-31-41-142 terraform-ec2]$
```